



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

Sviluppo Prime API

Angelo Sotgiu

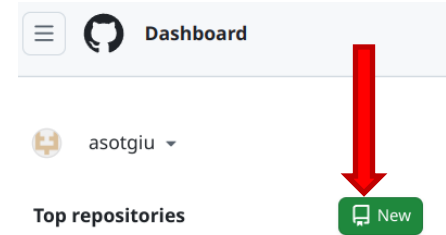
angelo.sotgiu@unica.it

In questa lezione

- Creazione e Configurazione di un repository GitHub
- Implementazione API del sistema biblioteca visto a lezione
- Validazione dei dati
- Metodi GET, POST, PUT e DELETE
- Documentazione automatica e test tramite documentazione

Creazione di un repository GitHub

- Se non lo avete già fatto, create un account su github.com
- Per prima cosa, creiamo un nuovo repository dall'interfaccia web di GitHub
- Configuriamo le opzioni disponibili...
 - nome del repository
 - descrizione
 - visibilità (pubblico/privato)
 - aggiunta README, .gitignore e licenza
 - > scegliamo il .gitignore configurato automaticamente per Python
- ...e clicchiamo su "Create Repository"
- Il nostro repository inizialmente sarà vuoto (a parte eventuali README, .gitignore e licenza)



Clonazione del repository GitHub

- Cliccando su "Code", possiamo visualizzare e copiare il link del repository



- Apriamo un terminale dentro la cartella dove vogliamo clonare il repository, e digitiamo:

`git clone <link del repository>`

- Avremo adesso una cartella locale con lo stesso nome del repository, che sarà automaticamente collegata al repository remoto. Possiamo vederne lo stato e i branch disponibili con i comandi:

`git status`

`git branch`

Configurazione del repository GitHub

- Possiamo aprire il progetto da un'IDE (es. PyCharm).
- Configuriamo adesso un file che specifica di che librerie ha bisogno il progetto, chiamato solitamente **requirements.txt**
- Quando creiamo un nuovo file, dobbiamo sempre dire a GitHub se tenerne traccia o meno:
 - se state usando un IDE, probabilmente vi verrà chiesto automaticamente
 - altrimenti da terminale usate questo comando:
git add <file>
se volete aggiungere tutti i file (usatelo con attenzione) -> git add .
- Nel file aggiungiamo le seguenti librerie:

```
fastapi[standard]  
requests
```

Creazione dell'environment

- Possiamo ora creare un nuovo environment python utilizzando virtualenv o conda. Da terminale (es. con conda):

```
conda create -n <nome_env> python=3.10  
conda activate <nome_env>
```

- Installiamo adesso i requisiti del progetto, sempre da terminale:

```
pip install -r requirements.txt
```

- Potremo aggiungere in seguito altri requisiti nel file requirements.txt
- Aggiorniamo il repository remoto! (questa operazione andrebbe eseguita di frequente)

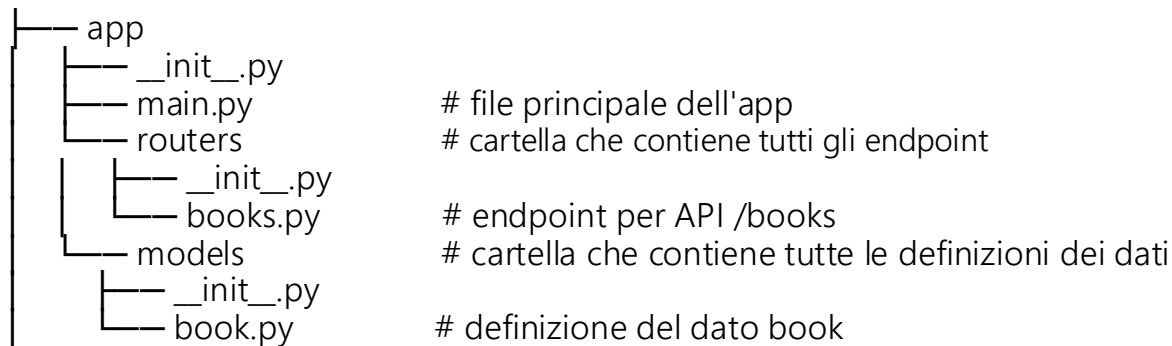
```
git commit -m "<commit_message>"  
git push
```

Riprendiamo l'esempio della biblioteca...

- Abbiamo bisogno di API per:
 - Ottenere la lista di libri -> GET /books
 - Ottenere il dettaglio di un singolo libro -> GET /books/{id}
 - Inserire una recensione per il libro scelto -> POST /books/{id}
- Per ogni libro, in questo caso d'uso specifico, ci serviranno gli attributi:
 - ID (intero)
 - Titolo (stringa)
 - Autore (stringa)
 - Recensione (intero tra 1 e 5)
- Possiamo implementare una classe che definisce la struttura dati "Book"
 - Useremo Pydantic, che farà automaticamente validazione dei dati contenuti nella struttura

Organizzazione del codice

- Prima di scrivere il codice, è meglio già pensare a come organizzarlo in maniera modulare, evitando di scrivere tutto dentro un file.
 - In questo modo sarà più facile capire, mantenere, testare e riutilizzare il codice.
- Non c'è uno standard unico, e le caratteristiche del progetto specifico possono orientare l'organizzazione.
- Usiamo quella consigliata da FastAPI, con qualche aggiunta:



Struttura dati per libri

- Definiamo la classe Book, ereditando dalla classe BaseModel di pydantic. È sufficiente definire gli attributi e il tipo di dato atteso. Potremmo anche definire dei valori di default.

```
from pydantic import BaseModel
```

```
class Book(BaseModel):  
    id: int  
    title: str  
    author: str  
    review: int = None
```

- Con questa implementazione non abbiamo però i vincoli sui valori della recensione

```
components:  
  schemas:  
    Book:  
      type: object  
      properties:  
        id:  
          type: integer  
          example: 1  
        title:  
          type: string  
          example: "Il nome della rosa"  
        author:  
          type: string  
          example: "Umberto Eco"  
        review:  
          type: integer  
          minimum: 1  
          maximum: 5  
          example: 5
```



Validazione avanzata dei dati

- Possiamo utilizzare il pattern di Python **Annotated** (importabile dalla libreria typing) per combinare. La sintassi è questa:

`Annotated[<type>, <metadata>]`

- `<type>` è il tipo di dato atteso (notazione standard type hints)
- `<metadata>` sono uno o più parametri che possono avere varie funzionalità
- Nel nostro caso, useremo la funzione `Field` di Pydantic, che accetta diversi parametri per applicare dei vincoli sui dati. Esempi:

`Field(ge=1, le=5)` # `ge`: greater than or equal to; `le`: less than or equal to

`Field(min_length=3)` # `min_length`: minimum length of the string

- Documentazione completa: <https://docs.pydantic.dev/latest/concepts/fields/>



Validazione avanzata dei dati

- Nel nostro caso quindi la nostra struttura dati Book diventerà:

```
from pydantic import BaseModel, Field
from typing import Annotated
```

```
class Book(BaseModel):
    id: int
    title: str
    author: str
    review: Annotated[int, Field(ge=1, le=5)] = None
```

Creazione di dati fittizi

- Simuliamo adesso un database che contiene già un elenco di libri. Creiamo un package **data** con un file books.py al suo interno, contenente un dizionario:

```
from models.book import Book
```

```
books = {  
    0: Book(id=0, title="libro zero", author="autore zero", review=3),  
    1: Book(id=1, title="libro uno", author="autore uno", review=5),  
    2: Book(id=1, title="libro due", author="autore due", review=1),  
}
```

- Attenzione: se l'import non funziona, è perché bisogna settare il path da cui l'interprete Python va a leggere i file. Potete farlo in due modi:
 - dall'IDE, impostate come "Sources root" la cartella app
 - da terminale, digitate

```
export PYTHONPATH=$PYTHONPATH:<path_cartella_app>
```

API /books

- Possiamo finalmente implementare la prima API! Ricordiamoci anche la documentazione!!!
- Al posto di farlo nel main.py, scriveremo il codice nel file **routers/books.py**

```
from fastapi import APIRouter  # usiamo questa classe al posto di FastAPI
from models.book import Book
from data.books import books
```

```
router = APIRouter(prefix="/books")  # tutti gli endpoint del router partiranno da /books
```

```
@router.get("/")
def get_all_books() -> list[Book]:
    """Returns the list of available books."""
    return list(books.values())
```



File principale dell'app

- Nel main, dobbiamo come sempre inizializzare l'applicazione web, includendo stavolta i router

```
from fastapi import FastAPI
from routers import books
```

```
app = FastAPI()
app.include_router(books.router, tags=["books"])
```

- Il campo "tags" è facoltativo, ma è utile per la documentazione:
diamoci uno sguardo -> <http://127.0.0.1:8000/docs>
- NB: per qualsiasi errore, il server risponde di default con status 500 e messaggio "Internal Server Error"



API /books/{id}

- Definiamo l'endpoint per la seconda API sfruttando i path parameter

```
@router.get("/{id}")
def get_book_by_id(id: int) -> Book:
    """Returns the book with the given ID."""
    return books[id]
```

- Posso anche aggiungere una descrizione per i parametri in questo modo:

```
from fastapi import APIRouter, Path
from typing import Annotated
```

```
@router.get("/{id}")
def get_book_by_id(
    id: Annotated[int, Path(description="The ID of the book to get")]
) -> Book:
    """Returns the book with the given ID."""
    return books[id]
```

API /books/{id}

- Dobbiamo ora gestire il caso in cui non esista un libro con l'id richiesto nel path:

```
from fastapi import APIRouter, Path, HTTPException
```

```
@router.get("/{id}")
def get_book_by_id(
    id: Annotated[int, Path(description="The ID of the book to get")]
) -> Book:
    """Returns the book with the given ID."""
    try:
        return books[id]
    except KeyError:
        raise HTTPException(status_code=404, detail="Book not found")
```

- Nella documentazione automatica purtroppo non viene incluso lo status 404

API /books/{id}/recensione

- Finora abbiamo sempre usato il metodo HTTP GET
 - le richieste (per convenzione) contengono parametri solo nell'URL (path e query parameter) e nell'header della richiesta HTTP
- Quando la richiesta HTTP necessita di contenere dati più corposi, si usa il metodo POST che (per convenzione) può contenerli nel **body** della richiesta.
 - possiamo sempre e comunque usare anche i parametri nell'URL e nell'header HTTP
- FastAPI permette di definire gli endpoint per il metodo POST in maniera molto semplice:
 - definiamo la struttura dati che accettiamo tramite un BaseModel Pydantic
 - usiamo il decoratore `@app.post("/path")`
 - definiamo una funzione di endpoint che prende come parametro la struttura dati che accettiamo, ed esegue internamente la logica necessaria

API /books/{id}/recensione

- Definiamo la struttura dati che accettiamo tramite un BaseModel Pydantic, dentro un nuovo file models/review.py

```
from pydantic import BaseModel, Field
from typing import Annotated

class Review(BaseModel):
    review: Annotated[int, Field(ge=1, le=5)]
```

API /books/{id}/recensione

- Creiamo la funzione di endpoint

```
@router.post("/{id}")
def add_review(
    id: Annotated[int, Path(description="The ID of the book to which add the
    review")],
    review: Review
):
    """Adds a review to the book with the given ID."""
    try:
        books[id].review = review.review
        return "Review successfully added"
    except KeyError:
        raise HTTPException(status_code=404, detail="Book not found")
    except ValidationError:
        raise HTTPException(status_code=400, detail="Not a valid request")
```

API /books - POST

- Possiamo adesso estendere l'applicazione in modo da prendere in input altri libri, accettando richieste POST sull'endpoint /books
- Per adesso scegliamo di non accettare inserimenti di libri il cui ID esiste già: per convenzione di usa il codice errore 403

```
@router.post("/")
def add_book(book: Book):
    """Adds a new book."""
    if book.id in books:
        raise HTTPException(status_code=403, detail="Book ID already exists")
    books[book.id] = book
    return "Book successfully added"
```



API /books/{id} - PUT

- Il metodo PUT viene usato per rimpiazzare completamente una risorsa

```
@router.put("/{id}")
def update_book(
    id: Annotated[int, Path(description="The ID of the book to update")],
    book: Book
):
    """Updates the book with the given ID."""
    if not book.id in books:
        raise HTTPException(status_code=404, detail="Book not found")
    books[id] = book
    return "Book successfully updated"
```

- Analogamente, il metodo PATCH può essere usato per rimpiazzare solo qualche campo della risorsa

API /books e /books/{id} - DELETE

- Possiamo anche definire dei metodi per eliminare un libro archiviato, o tutti quanti

```
@router.delete("/")
def delete_all_books():
    """Deletes all the stored books."""
    books.clear()
    return "All books successfully deleted"
```

```
@router.delete("/{id}")
def delete_book(
    id: Annotated[int, Path(description="The ID of the book to delete")]
):
    """Deletes the book with the given ID."""
    try:
        del books[id]
        return "Book successfully deleted"
    except KeyError:
        raise HTTPException(status_code=404, detail="Book not found")
```

Query Parameters

- Solitamente sono utilizzati per specificare delle opzioni su come la risposta verrà fornita
Es.: filtraggio, ordinamento, etc.
- Aggiungiamo un parametro query sull'API /books per ordinare libri per valore recensione

```
@router.get("/")
def get_all_books(
    sort: Annotated[bool, Query(description="Sort books by their review")] = False
) -> list[Book]:
    """Returns the list of available books."""
    if sort:
        return sorted(books.values(), key=lambda book: book.review)
    else:
        return list(books.values())
```